

$([\wedge\&=]^+)=([\wedge\&=]^*)$

# Reguläre Ausdrücke



# Reguläre Ausdrücke

Reguläre Ausdrücke (auch Regex) sind **Muster** oder Schablonen, die verwendet werden, um **Text** zu **durchsuchen**, zu **analysieren** oder zu **manipulieren**.

Regex sind **plattformunabhängig**, d. h., sie sind in vielen verschiedenen Programmiersprachen und Apps verfügbar.

Es gibt viele Anwendungsgebiete wie z. B. Benutzereingaben prüfen, Textsuche, Text ersetzen, **Parsen** von Dateien oder Webseiten.

**Parsen** bezeichnet das Analysieren und Strukturieren von Text nach festen Regeln, z. B. zur Weiterverarbeitung in Programmen.

# Regex-Beispiele 1/11

```
String result = "Hello Adele!".replaceAll("[Ee]!", "🤖");  
System.out.println(result);
```

**Ausgabe:**

H🤖lo Ad🤖e!

# Regex-Beispiele 2/11

```
String result = "Hello Adele!".replaceAll("[a-z]", "🤖");  
System.out.println(result);
```

**Ausgabe:**

H🤖🤖🤖🤖 A🤖🤖🤖🤖 !

# Regex-Beispiele 3/11

```
String result = "Hello Adele!".replaceAll("[^a-z]", "🤖");  
System.out.println(result);
```

Ausgabe:

🤖ello🤖dele🤖

# Regex-Beispiele 4/11

```
String regex = "H...o.A..l.";
String result = "Hello Adele!".replaceAll(regex, "👾");
System.out.println(result);
```

**Ausgabe:**



# Regex-Beispiele 5/11

```
String result = "HiHi Hi Adele!".replaceAll("Hi\\s?", "👾");  
System.out.println(result);
```

**Ausgabe:**

👾👾👾 Adele!

# Regex-Beispiele 6/11

```
String result = "HiHi Hi Adele!".replaceAll("Hi\\s", "🤖");  
System.out.println(result);
```

**Ausgabe:**

Hi🤖🤖 Adele!



# Regex-Beispiele 7/11

```
String result = "Hello Adele!".replaceAll("|+", "🤖");  
System.out.println(result);
```

**Ausgabe:**

He🤖 o Ade🤖 e!

# Regex-Beispiele 8/11

```
String result = "Hello Adele!".replaceAll(".*dele.*", "🤖");  
System.out.println(result);
```

Ausgabe:



## Achtung:

Das \* in Kombination mit String::replaceAll führt unter bestimmten Umständen zu ungewollten zusätzlichen Zeichen (siehe nächste Folie).

# replaceAll mit \*

Ausgabe:

Heo Adee!

String result = "Hello Adele!".replaceAll("l\*", ""); // Das Sternchen führt zu ungewollten Zusatz-  
"Ersetzungen"

Ein regulärer Ausdruck berücksichtigt immer auch den Anfang ^ und das Ende \$ einer Zeichenkette.

Bei der Auswertung des regulären Ausdrucks wird die Zeichenkette zeichenweise durchlaufen und geprüft. Hier trifft der Ausdruck l\* auf jedes Zeichen zu, allerdings oft mit 0 Zeichen, was dazu führt, dass kein Zeichen ersetzt wird (da 0 l vorkommen). Trotzdem hängt die Methode String::replaceAll ein zusätzliches 🤖 an. Taucht im Match allerdings ein l auf, so wird das l durch 🤖 ersetzt.

| Position | Zeichen       | l* match?                       | Ergebnis                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------|---------------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0        | (Anfang ^)    | ja (0 l), eine Hinzufügung      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 1        | H             | ja (0 l), eine Hinzufügung      |  H                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2        | e             | ja (0 l), eine Hinzufügung      |  H  e                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 3 und 4  | l             | ja (1 l), <b>eine Ersetzung</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|          |               | ja (1 l), <b>eine Ersetzung</b> |  H  e                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 5        | o             | ja (0 l), eine Hinzufügung      |  H  e   o                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 6        | (Leerzeichen) | ja (0 l), eine Hinzufügung      |  H  e                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 7        | A             | ja (0 l), eine Hinzufügung      |  H  e     A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 8        | d             | ja (0 l), eine Hinzufügung      |  H  e     A  d                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 9        | e             | ja (0 l), eine Hinzufügung      |  H  e     A  d  e                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 10       | l             | ja (1 l), <b>eine Ersetzung</b> |  H  e     A  d  e                                                                                                                                                                                                                                                                                                                                                              |
| 11       | e             | ja (0 l), eine Hinzufügung      |  H  e     A  d  e   e                                                                                                                                                                                                                                                                     |
| 12       | !             | ja (0 l), eine Hinzufügung      |  H  e     A  d  e    e  !                                                                                       |
| 13       | (Ende \$)     | ja (0 l), eine Hinzufügung      |  H  e     A  d  e    e  !  |

# Regex-Beispiele 9/11

```
String result = "Deinem Adele."  
    .replaceAll("Dein(e|em|er)", "👾");  
System.out.println(result);
```

**Ausgabe:**  
👾m Adele.

# Regex-Beispiele 9/11 (Korrekt)

```
String result = "Deinem Adele."  
    .replaceAll("Dein(em|er|e)", "👾");  
System.out.println(result);
```

**Ausgabe:**

👾 Adele.

# Regex-Beispiele 10/11

```
String result = "6\\2=3".replaceAll("\\d", "🤖");  
System.out.println(result);
```

**Ausgabe:**

🤖 \ 🤖 = 🤖

# Regex-Beispiele 11/11

```
String result = "6\\2 = 3".replaceAll("\\\\", "÷");  
System.out.println(result);
```

**Ausgabe:**

6÷2 = 3

# Escape-Zeichen \

In regulären Ausdrücken gibt es bestimmte **Escape-Zeichen**, die eine **besondere Bedeutung** haben, wie zum Beispiel das \ (Backslash).

In Java-Strings wird das **Backslash als Escape-Zeichen** verwendet, um bestimmte Zeichen wie \" oder nicht-sichtbare Zeichen wie \n und \t zu repräsentieren.

Ein Backslash muss daher in einem String als \"\" **escaped** werden, damit es als einfaches \-Zeichen dargestellt wird.

Außerdem wird das Backslash auch zum Escapen (z. B. \"\".\" oder \"\"\*\") in regulären Ausdrücken verwendet, damit spezielle Regex-Zeichen wie . oder \* ihre reguläre Bedeutung verlieren und stattdessen als einfaches Zeichen dargestellt werden.

Aufgrund dieser beiden Gründe muss in einem regulären Ausdruck in einem Java-String das Backslash durch \"\"\"\" ausgedrückt werden:

```
String result = "6\\2 = 3".replaceAll("\\\\", "÷");
```

↑  
Java-String

↑  
Regex im Java-String

Der Compiler wandelt die Strings um zu: "6\2 = 3".replaceAll("\\", "÷");



# Regex-Elemente (Auswahl)

|                              |                                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------|
| .                            | Beliebiges einzelnes Zeichen außer Zeilenumbruch (" <code>\n</code> ").                                        |
| *                            | Beliebig viele Wiederholungen (auch keine) des vorhergehenden Zeichens.                                        |
| +                            | Mindestens eine Wiederholung des vorhergehenden Zeichens.                                                      |
| {n}                          | Genau n-Mal X                                                                                                  |
| {n,m}                        | Zwischen n- und m-Mal X                                                                                        |
| ?                            | Optional: Keine oder genau eine Wiederholung des vorhergehenden Zeichens.                                      |
| []                           | Ein Zeichen aus einer Zeichenklasse, z. B. [a-z] für alle Kleinbuchstaben.                                     |
| [^]                          | Negation. Kein Zeichen aus der Klasse, z. B. [^0-9] für kein Ziffernzeichen.                                   |
| ()                           | Eine Sequenz von Zeichen. Eine Gruppierung ermöglicht deren Wiederholung.                                      |
|                              | Logisches Oder, z. B. a b für a oder b.                                                                        |
| ^                            | Anfang der Zeile                                                                                               |
| \$                           | Ende der Zeile                                                                                                 |
| \                            | Escaping von Metazeichen, z. B. \. für einen Punkt.                                                            |
| \d, \w, \s und<br>\D, \W, \S | \d (Ziffer), \w (Wortzeichen), \s (Leerzeichen, Tab, Zeilenumbruch) und ihre negierten Versionen (\D, \W, \S). |
| \b                           | Wortgrenze, z. B. \bword\b stellt sicher, dass nur word gefunden wird, aber nicht sword.                       |

# String::replaceAll



Die Klasse String enthält einige Methoden, die Regex eingebaut haben.

String::replaceAll nimmt einen Regex entgegen, um alle Übereinstimmungen mit einem anderen String zu ersetzen.

```
String result = "Hello Adele! Du bist stark.".replaceAll("[^\\w|\\s]", "");  
System.out.println(result);
```

**Ausgabe:**

Hello Adele👾 Du bist stark👾

# String::split



Teilt eine Zeichenkette überall, wo der Regex übereinstimmt.

```
String[] result = "Hello Adele! Du bist stark.".split(" "); // result.length == 5  
System.out.println(Arrays.toString(result));
```

## Ausgabe:

```
[Hello, Adele!, Du, bist, stark.]
```

# String::split mit limit



Mit dem zweiten, optionalen Parameter *limit* kann man die Anzahl der Elemente im Ergebnis-Array begrenzen.

```
String[] result = "Hello Adele! Du bist stark.".split(" ", 3); // result.length == 3
System.out.println(Arrays.toString(result));
```

## Ausgabe:

```
[Hello, Adele!, Du bist stark.]
```

# String::matches



Die Methode `String::matches` prüft, ob ein regulärer Ausdruck mit einer Zeichenkette übereinstimmt.

`boolean` result;

```
result = "Hello Adele! Du bist stark.".matches("Adele.*bist stark.$");  
System.out.println(result); // false => String beginnt nicht mit Elona
```

```
result = "Hello Adele! Du bist stark.".matches(".*Adele.*bist stark.$");  
System.out.println(result); // true => Vor Elona sind beliebige Zeichen.
```

**Ausgabe:**

```
false  
true
```

# Weitere nützliche String-Methoden (Auswahl)

Folgende String-Methoden können nützlich beim Parsen sein, auch wenn sie nicht direkt mit Regex im Zusammenhang stehen.

```
"hallo".substring(1, 4); // "all"
"  hallo ! ".trim(); // "hallo !"
"hallo".indexOf("al"); // 1
"hallo".contains("all"); // true
"hallo".startsWith("ha"); // true
"hallo".endsWith("llo"); // true
"".isEmpty(); // true
" ".isEmpty(); // false
" ".isBlank(); // true
"hallo".charAt(1); // 'a' (Rückgabewert ist char)
Character.isLetter('a'); // true
Character.isDigit('1'); // true
Character.isSpaceChar(' '); // true
```

FYI: Mit **formatted** lassen sich relativ übersichtlich längere Strings mit Variablen zusammensetzen, wobei die Sonderzeichen %s und %d als Platzhalter dienen.

```
"Hallo %s, deine Lieblingszahl ist %d.".formatted("Sara", 42); //
Hallo Sara, deine Lieblingszahl ist 42.
```

FYI: Java unterstützt **Multiline-Strings**, die mit einem dreifachen " beginnen, gefolgt von einem Zeilenumbruch:

```
System.out.println(""" ← Muss am Ende einer Zeile stehen.
    Erste Zeile
    Zweite Zeile
    Dritte Zeile""");
```

# Pattern und Matcher

Die Methoden `String::replaceAll`, `String::split` und `String::matches` verwenden intern die Klassen **Pattern** und **Matcher** für die Auswertung der regulären Ausdrücke.

Ein regulärer Ausdruck wird zunächst mit der Klasse **Pattern** (Muster) kompiliert (optimiert). Das Kompilieren sollte aus Performance-Gründen nur einmalig erfolgen (nicht in Schleifen).

Die Klasse **Pattern** hat die Methode **matcher(String)**, die einen Eingabe-String erhält und einen **Matcher** zurückgibt.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("A(\\w)*d", Pattern.CASE_INSENSITIVE);  
    String input = "Android Ad Abenteuer bald Abend";  
    Matcher matcher = pattern.matcher(input);  
    while (matcher.find()) { // prüft, ob es einen weiteren Match gibt (true/false).  
        System.out.println(matcher.group()); // group() gibt den nächsten Match als String zurück.  
    }  
}
```

**Ausgabe:**  
Android  
Ad  
ald  
Abend

```
System.out.println(matcher.group()); // IllegalStateException: No match found → find() gibt false zurück.  
}
```

# Pattern und Matcher: Groups

Mit runden Klammern lassen sich Gruppen im regulären Ausdruck definieren, die sich durch Indizes über `Matcher::group` abrufen lassen.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("(\\d{3})-(\\d{2})-(\\d{4})");  
    String input = "123-45-6789";  
    Matcher matcher = pattern.matcher(input);  
    if (matcher.matches()) { // prüft, ob der input als Ganzes matched (true/false).  
        System.out.println("Full match: " + matcher.group(0));  
        System.out.println("Group 1: " + matcher.group(1));  
        System.out.println("Group 2: " + matcher.group(2));  
        System.out.println("Group 3: " + matcher.group(3));  
    }  
}
```

## Ausgabe:

```
Full match: 123-45-6789  
Group 1: 123  
Group 2: 45  
Group 3: 6789
```

```
Matcher matcher2 = pattern.matcher("abc 123-45-6789 xyz");  
matcher2.matches(); // false → der ganze Text passt nicht aufs Muster (Pattern).  
matcher2.find();    // true → weil ein Teilstring ("123-45-6789") passt.  
}
```



# Non-Capturing Groups

Wenn runde Klammern () in einem regulären Ausdruck verwendet werden, erzeugt das eine **Capturing Group**, die sich mittels `matcher.group(n)` abfragen lässt.

Wenn die Klammerung **nur für die Gruppierung der Zeichen, aber nicht zur Speicherung** verwendet werden soll, wird eine sogenannte **non-capturing group** benötigt, die **mit ?: beginnt**.

```
public static void main(String[] args) {  
    Pattern pattern = Pattern.compile("(?:\\d+)-(abc)+");  
    String input = "123-abcabc";  
    Matcher matcher = pattern.matcher(input);  
    if (matcher.matches()) {  
        System.out.println(matcher.group(1)); // gibt abc aus. Ließe man das ?: weg, würde man stattdessen 123 erhalten.  
    }  
}
```

- `(?:\\d+)` wird nur für das Regex + gruppiert, ist aber **nicht** mittels `group(1)` abrufbar.
- `(abc)+` ist eine Capturing Group und kann mittels `group(1)` abgerufen werden.

**Achtung:** Bei einer Wiederholung (+, \*, {n,m}) gibt `group(n)` **den letzten erfolgreichen Match** der Gruppe zurück. Will man stattdessen den gesamten Teil "abcabc" erhalten, muss man `"(?:\\d+)-((?:abc)+)"` als Regex verwenden.

# Weitere Regex-Beispiele

# Beispiel A 1/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

# Beispiel A 2/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

# Beispiel B 1/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

# Beispiel B 2/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies“

Match: „Der **fiese** friesische Fliesenleger fliest mit **fies**em friesischem Fliesenkleber“.

# Beispiel C 1/4

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies(e|er|es|en|em)“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

# Beispiel C 2/4

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies(e|er|es|en|em)“

Match: „Der **fiese** friesische Fliesenleger fliest mit **fiesem** friesischem Fliesenkleber“.

**Warum wird „fiesem“ nicht komplett gefunden?**



# Beispiel C 3/4

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies(e|er|es|en|em)“

Match: „Der **fiese** friesische Fliesenleger fliest mit **fiesem** friesischem Fliesenkleber“.

**Warum wird „fiesem“ nicht komplett gefunden?**

**Der reguläre Ausdruck wird von links nach rechts interpretiert und entsprechend nach dem Pattern gesucht. Ein „e“ wird also zwangsläufig immer vor einem „em“ gefunden.**

# Beispiel D 4/4

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF]ies(em|er|es|en|e)“

Match: „Der **fiese** friesische Fliesenleger fliest mit **fiesem** friesischem Fliesenkleber“.

# Beispiel D 1/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF][lr]?ies“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

# Beispiel D 2/2

Text: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.

Regex: „[fF][lr]?ies“

Match: „Der fiese friesische Fliesenleger fliest mit fiesem friesischem Fliesenkleber“.